

Conduire avec les yeux

Grâce à un système de navigation autonome

Younes EL MRABET Jules GATELIER Dylan GIRAULT
Hugo HUYET-DUMONG Félix LIBURSKI

19 août 2026

Résumé

Ce rapport d'étude présente le projet "Conduire avec les yeux" de l'UV SY27/AI36 de l'UTC pour le semestre A25, ainsi qu'un état de l'art de la conduite autonome et nos solutions pour parvenir à réussir ce challenge.

1 Introduction

Les systèmes de conduite autonome n'ont pas encore atteint la fiabilité nécessaire pour fonctionner sans supervision humaine. Le conducteur doit donc rester en permanence dans la boucle de contrôle et conserver l'autorité finale sur les décisions du véhicule. Dans ce contexte, les constructeurs doivent être capables de vérifier en continu que le conducteur est engagé dans la tâche de conduite et qu'il peut reprendre la main à tout moment. Dans le cadre du challenge proposé dans l'UV SY27, chaque équipe doit développer un pipeline combinant (1) un module de perception LiDAR pour l'estimation de l'espace navigable, (2) un système de monitoring du conducteur basé sur la vision, et (3) un comportement global conditionnant l'autonomie du véhicule à l'engagement humain, tout en permettant une coopération explicite entre le véhicule et son conducteur.

Le rapport décrit l'architecture retenue, les solutions techniques étudiées pour chaque module, les choix méthodologiques effectués et le planning de développement associé. Chaque membre de l'équipe est responsable d'au moins une brique fonctionnelle et contribue à la rédaction de la section correspondante.

2 Architecture

Le Manager est le premier module à s'allumer. Il initialise les autres packages selon le mode choisi (mode normal, calibration, etc.). Une fois l'initialisation effectuée, il lance le cycle principal du système.

Le cycle est organisé de la manière suivante :

- Perception intérieure et Perception extérieure démarrent simultanément en parallèle.
- La Perception extérieure envoie ses résultats au module Localisation, qui estime la pose du véhicule, puis transmet cette pose au module Navigation.
- En parallèle, la Perception intérieure envoie directement ses sorties au module Navigation.
- Le module Navigation fusionne les informations provenant de la perception intérieure et extérieure pour obtenir une commande de trajectoire.
- Cette commande est ensuite transmise au module Contrôle, qui calcule les actions physiques à appliquer (direction, accélération, freinage).
- Une fois son traitement terminé, le module Contrôle envoie un signal de fin au Manager, qui peut alors relancer un nouveau cycle.

Ainsi, le Manager assure la synchronisation du système tout en laissant les modules fonctionner de manière autonome et optimisée.

3 Perception intérieure

La perception intérieure vise à donner au véhicule la capacité de comprendre les intentions, et l'état du conducteur en analysant des signaux visuels (visage, main, posture). Dans ce projet, l'objectif est

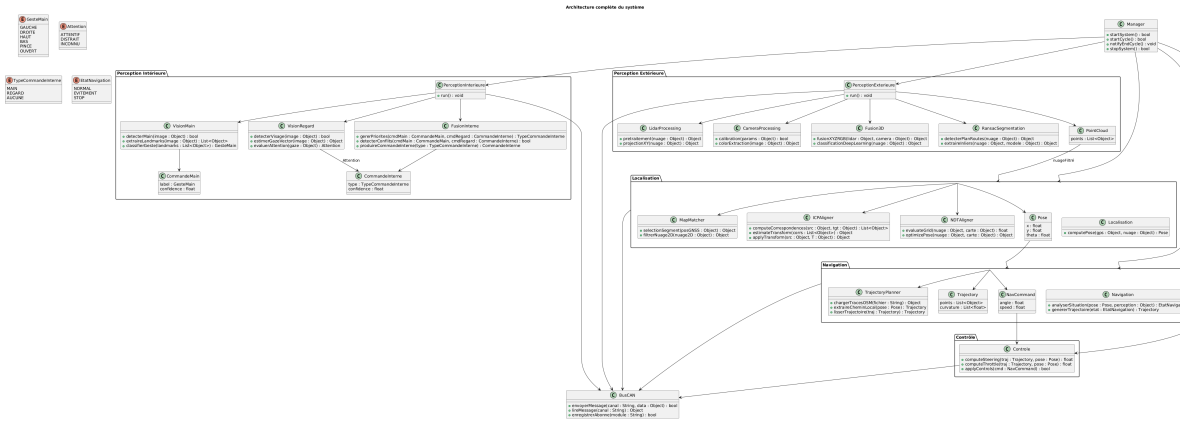


FIGURE 1 – Diagramme UML de l'architecture du système

de permettre au conducteur de guider le véhicule sans volant en combinant à travers les gestes de la main mais également de vérifier son niveau de concentration en analysant plusieurs signaux de fatigue et de distraction.

3.1 Approche globale

La perception intérieure est organisée en 3 modules. Le premier se concentre sur la concentration du conducteur. Le but est d'analyser certains points du visage et d'en déduire un score de fatigue et de distraction qui va donner un score de concentration. Le second module permet de détecter les gestes intentionnels de conduite du conducteur pour guider la voiture. Ce module se concentre sur la main du conducteur et permet d'ordonner les principales informations à la voiture comme démarrer ou arrêter la voiture ou tourner à droite ou à gauche. Enfin, le dernier module fusionne les informations de concentration du conducteur et les gestes de conduite pour envoyer les bonnes indications au module de navigation de la voiture.

3.2 Architecture

3.3 Concentration du conducteur

3.4 Détection des gestes

3.5 Contexte et objectifs

La perception intérieure vise à donner au véhicule la capacité de comprendre l'intention, l'attention et l'état du conducteur en analysant des signaux visuels (regard, gestes, posture, expressions). Dans ce projet, l'objectif est de permettre au conducteur de guider le véhicule sans volant en combinant deux modalités naturelles : le regard et les gestes de la main.

3.6 La conduite

Voici le raisonnement qui a guidé le choix des deux modalités et l'ordre d'intégration (gestes puis regard) :

3.6.1 La recherche de l'interaction la plus naturelle et fluide : le regard.

Le regard est l'interface la plus directe entre l'intention mentale et l'action perçue. Regarder une direction est souvent le premier acte cognitif lorsqu'on souhaite s'orienter ou diriger l'attention vers un objet. En essayant de concevoir la conduite la plus intuitive possible, l'idée d'utiliser le regard apparaît naturellement : c'est une interaction sans friction, sans contact, et qui peut être très rapide.

Si la reconnaissance du regard est fiable et sûre, elle permet une conduite quasiment "transparente" :

le conducteur n'a pas besoin d'effectuer d'action supplémentaire, il suffit de regarder. C'est aussi une étape intéressante vers un véhicule 100% autonome puisque la machine apprend à associer une intention visuelle à une action de conduite.

3.6.2 La nécessité d'une étape transitoire : les gestes de la main.

Passer directement d'une conduite au volant à une conduite contrôlée uniquement par le regard peut être déroutant et créer une perte de confiance pour certains conducteurs. Le geste apporte une preuve explicite d'intention : un pouce pointé, une paume levée, constituent des signaux délibérés et facilement interprétables.

Les gestes servent donc de pont psychologique et fonctionnel : ils permettent aux conducteurs d'adopter progressivement une nouvelle façon d'interagir avec le véhicule, tout en conservant une action volontaire et visible qui valide l'intention. Dans la transition vers le regard, les gestes peuvent jouer plusieurs rôles pratiques : apprentissage, confirmation explicite.

3.6.3 Ordre d'intégration (Phase 1 : gestes → Phase 2 : regard).

Commencer par la détection des gestes permet d'établir une base robuste de contrôle explicite, simple à évaluer et à sécuriser. Une fois que ce canal fonctionne de manière fiable, on peut intégrer la perception du regard pour améliorer l'intuitivité et réduire l'effort physique du conducteur.

Le geste doit rester prioritaire : il constitue une preuve d'intention volontaire. Le regard complète et enrichit la compréhension du comportement du conducteur (confirmation, attention, déclencheurs automatiques), mais sans jamais remplacer la garantie fournie par une action volontaire tant que l'on est dans une phase de transition.

3.7 Logique d'implémentation et scénarios d'interaction conducteur-véhicule

3.7.1 Déroulé typique d'une session de conduite

1. Mise en marche :

Le conducteur montre un pouce vers le haut pour signaler au véhicule de démarrer. Le système vérifie que le regard est dirigé vers la route et que l'attention est stable avant d'exécuter la commande.

2. Conduite en ligne droite :

Tant qu'aucun geste n'est détecté et que le regard reste orienté vers la route, le véhicule maintient son cap. Le module d'attention surveille l'état du conducteur (yeux ouverts, regard actif, absence de distraction prolongée).

3. Changement de direction :

- Le conducteur effectue un geste de pouce gauche ou droit pour indiquer une intention de tourner.
- Le regard vient confirmer cette intention : le conducteur regarde brièvement dans la direction du virage.
- Le module décisionnel valide la commande et déclenche la manœuvre.

4. Arrêt du véhicule :

Le conducteur présente une main ouverte (signe stop). Le système arrête le véhicule immédiatement, indépendamment de la position du regard.

5. Reprise après arrêt :

Le véhicule reste à l'arrêt tant qu'il ne reçoit pas un pouce vers le haut indiquant le redémarrage.

6. Inattention détectée :

Si le conducteur détourne le regard de la route pendant une durée supérieure au seuil critique, le véhicule déclenche une alerte visuelle ou sonore, puis un arrêt de sécurité si l'inattention persiste.

3.7.2 Logique décisionnelle macro

Le comportement global du système peut être représenté par un enchaînement d'états :

1. État d'attente → attente d'un geste "pouce haut".
2. État de conduite → lecture continue du regard et des gestes.
3. État d'action → exécution d'une commande validée.
4. État de sécurité → en cas de perte de détection, d'inattention ou d'incertitude.

Chaque module (vision main, vision regard, décision) travaille en parallèle :

- **Module Main** → produit une commande explicite + un score de confiance.
- **Module Regard** → produit une orientation + un score d'attention.
- **Module Décisionnel** → combine les sorties selon des règles de priorité :
 - Geste prioritaire.
 - Regard comme support décisionnel ou surveillance.
 - Sécurité toujours prioritaire sur toute autre action.

3.8 Architecture technique générale du système

3.8.1 Vue d'ensemble du système

L'architecture technique du système repose sur une structure modulaire et hiérarchique permettant d'assurer la robustesse et l'évolutivité de la perception intérieure. Elle est organisée en trois grandes couches :

1. **Couche de perception** : acquisition et traitement des données visuelles (main, regard, visage).
2. **Couche d'interprétation** : extraction et classification des intentions du conducteur.
3. **Couche de décision et d'action** : fusion des informations, validation des intentions, et envoi des commandes au module de navigation du véhicule.

3.9 Architecture modulaire détaillée

3.9.1 Module de perception des gestes de la main

- **Entrée** : flux vidéo RGB (caméra tableau de bord).
- **Détection** : détection de la main.
- **Reconnaissance des gestes** :
 - Pouce gauche → tourner à gauche
 - Pouce droit → tourner à droite
 - Pouce haut → démarrage
 - Main ouverte → stop
- **Sorties** : label de geste, score de confiance, timestamp.

3.9.2 Module de perception du regard et de l'attention

- **Entrée** : flux vidéo RGB (caméra tableau de bord).
- **Détection** : détection d'un visage, yeux, iris.
- **Projection du regard** :
 - Regard gauche → intention de tourner à gauche
 - Regard à droite → intention de tourner à droite
 - Regard droit → conducteur concentré
 - Aucune détection du regard ou regard distrait (hors de la route) → warning + stop
- **Sorties** : label de geste, score de confiance, timestamp.

3.9.3 Module décisionnel

- **Entrées** : sorties du module main + module regard.
- **Fonctions principales** :
 - Priorisation des gestes sur le regard.
 - Interprétation contextuelle.
 - Validation de la cohérence geste/regard.
 - Gestion des états du conducteur.

- **Sorties :**
 - Tourner à droite
 - Tourner à gauche
 - Arrêter le véhicule
 - Redémarrer le véhicule
 - Warning (manque d'attention ou commande invalide)

3.9.4 Stratégie d'intégration Phase 1 → Phase 2

- **Phase 1 (gestes) :** pipeline main → décision → véhicule.
- **Phase 2 (regard) :** ajout du pipeline regard sans modifier le pipeline geste.
- **Transition fluide :** pipelines parallèles avec priorisation du geste.

3.10 Benchmark des approches de perception des gestes de la main pour la conduite

La reconnaissance de gestes est un élément fondamental pour un véhicule sans volant. Deux familles d'approches existent :

1. classification directe d'images,
2. analyse géométrique à partir de landmarks.

3.10.1 Approche 1 — Classification directe d'images

Principe général : Un CNN reçoit une image contenant la main et prédit directement la classe du geste.

Modèles typiques :

- CNN légers (MobileNetV2, EfficientNet-Lite)
- CNN classiques (ResNet, VGG)
- Transformers compressés (MobileViT, DeiT-Tiny)
- Détecteurs end-to-end (YOLO gestuel)

Avantages :

- Simplicité conceptuelle
- Flexibilité
- Indépendance vis-à-vis des landmarks
- Domaine scientifique mature

Limites :

- Besoin d'un dataset important
- Sensible aux conditions réelles
- Coût computationnel élevé
- Long à entraîner

3.10.2 Approche 2 — Landmarks + règles géométriques

Principe général : Détection de 21 points clés de la main puis classification via :

- angles des phalanges,
- direction du pouce,
- ouverture de la main,
- distances relatives,
- orientation de la paume.

Avantages :

- Ultra-rapide, léger
- Robuste à la lumière
- Pas besoin de dataset
- Fiable pour gestes simples
- Interprétable

Limites :

- Moins efficace pour gestes complexes
- Dépend de la qualité des landmarks
- Nécessite des règles manuelles

3.10.3 Contraintes du projet : justification du choix landmarks + géométrie

Contraintes systèmes :

- faible consommation CPU,
- robustesse inter-utilisateur,
- fonctionnement > 30 FPS,
- sécurité.

Exigences :

- légèreté computationnelle,
- invariance morphologique,
- robustesse,
- absence d'entraînement.

3.11 Benchmark des technologies de hand pose estimation

Dans le cadre de l'estimation de la pose de la main en temps réel, deux familles de modèles ressortent systématiquement dans la littérature et dans les implémentations industrielles : **MediaPipe Hands**, développé par Google, et les architectures **YOLOv8-Pose** spécialisées pour la détection de keypoints de la main. Ces deux approches dominent en raison de leur maturité, de leurs performances et de leur capacité à fonctionner dans des pipelines temps réel, notamment en environnement embarqué.

3.11.1 MediaPipe Hands

MediaPipe Hands est l'une des solutions les plus largement utilisées pour la détection et le suivi de la main. Ce modèle repose sur une architecture en deux étapes : un détecteur de paume (*BlazePalm*), suivi d'un réseau prédictif de 21 landmarks 3D de la main. Cette architecture permet d'obtenir un système léger, robuste et particulièrement efficace sur mobiles et dispositifs embarqués.

Caractéristiques principales

- **Architecture** : Pipeline "détection de paume + prédiction de landmarks".
- **Landmarks** : 21 points 3D (coordonnées x , y , profondeur relative z) [9].
- **Performance temps réel** : optimisé pour fonctionner sur CPU ou GPU mobile avec une latence minimale, grâce à la fusion détection + suivi [11].
- **Robustesse** : bonne stabilité face aux variations d'orientation, d'échelle et aux occlusions partielles, notamment grâce au recadrage dynamique centré sur la paume.
- **Données d'entraînement** : environ 30 000 images réelles complétées par des données synthétiques pour la profondeur [8].
- **Intégration** : prise en charge native dans MediaPipe (Android, iOS, Web, Python), permettant une intégration rapide dans un pipeline embarqué.

3.11.2 YOLOv8-Pose pour la main

YOLOv8-Pose est une architecture générique de pose estimation qui peut être entraînée sur des jeux de données spécifiques pour la main. Ultralytics propose notamment un dataset annoté de 21 keypoints de la main, compatible avec YOLOv8-Pose [13]. Plusieurs travaux académiques ont montré que YOLOv8-Pose, entraîné sur un dataset de mains, offre de bonnes performances de détection et de localisation des keypoints [kadam2024yolov8].

Caractéristiques principales

- **Landmarks** : jusqu'à 21 keypoints correspondant aux articulations standards de la main [13].
- **Flexibilité du modèle** : plusieurs tailles disponibles (nano, small, medium, large, extra-large), permettant de choisir un compromis entre précision et coût d'inférence.
- **Précision** : performances élevées en termes de catégorie détectée (main) et en termes de localisation des keypoints (mAP).
- **Entraînement** : architecture entièrement ré-entraînable et adaptable à des scénarios personnalisés (mains de conducteurs, gestes spécifiques, etc.).

3.11.3 Comparaison et analyse

MediaPipe Hands et YOLOv8-Pose sont deux approches pertinentes, mais elles reposent sur des philosophies opposées.

- **MediaPipe Hands** privilégie la *légèreté*, la *latence faible* et la *robustesse en conditions mobiles*. Son pipeline hybride détection + suivi est extrêmement efficace pour un système embarqué.
- **YOLOv8-Pose**, à l'inverse, est plus *modulaire* et *entraînable*. Il peut être optimisé pour un domaine spécifique (comme la détection de gestes spécifiques), au prix d'un coût d'inférence supérieur.

Synthèse comparative

- Si l'objectif principal est la **latence minimale** dans un système embarqué, MediaPipe Hands constitue la solution la plus adaptée.
- Le système dispose d'un **GPU embarqué**, YOLOv8-Pose reste une alternative intéressante
- Pour des scénarios nécessitant une grande flexibilité (plusieurs poses, multi-personne, détection générique), YOLOv8-Pose est plus extensible.

3.11.4 Conclusion

En raison de leurs performances, de leur maturité et de leurs cas d'usage variés, MediaPipe Hands et YOLOv8-Pose sont les deux approches qui ressortent le plus souvent dans la littérature et les projets industriels d'estimation de pose de la main. Leur choix dépend principalement du contexte matériel et des besoins en précision ou en personnalisation du modèle. Par nécessité d'implémentation rapide et à la recherche d'optimisation de la puissance de calcul, nous avons priorisé sur MediaPipe Hands.

3.11.5 Plan B : Classification d'images

Quand basculer ? En cas de difficulté à détecter les gestes de la main à partir des positions géométriques des landmarks de la main, nous essayerons l'approche par classification.

Mise en œuvre :

1. Utilisation ou création d'un dataset
2. Entraînement d'un modèle léger
3. Intégration hybride (détection + crop + classification)

3.12 Benchmark des approches de suivi du regard pour la conduite

3.12.1 Introduction et enjeux

Dans le contexte d'un système embarqué, plusieurs contraintes majeures apparaissent : (1) puissance de calcul limitée, (2) variations lumineuses importantes (jour, nuit, arrière-plans dynamiques),

(3) position du conducteur variable, (4) nécessité d’une exécution fiable en temps réel (30–60 FPS). L’objectif principal est donc de construire un module robuste, léger et indépendant de calibrations complexes.

3.12.2 Panorama des approches de suivi du regard

Estimation du regard L’approche directe consiste à fournir une image du visage ou de l’œil à un réseau de neurones, qui prédit directement la direction du regard, sous la forme d’un vecteur 3D. Cette approche obtient d’excellents résultats dans des environnements contrôlés, mais cette approche dépendant d’un dataset calibré, est relativement sensible à l’éclairage et l’occlusion, a coût de calcul assez élevé.

Approche landmarks + estimation géométrique L’approche indirecte repose sur la prédiction des points clés du visage (landmarks), puis sur l’analyse géométrique pour estimer l’orientation de la tête et la direction du regard. Cette stratégie a été popularisée par les travaux de Google Research avec MediaPipe FaceMesh [14], qui permet d’extraire 468 points du visage à très haute fréquence.

Elle consiste à : (1) détecter les landmarks du visage, (2) extraire les points des yeux et de l’iris, (3) calculer l’orientation de la tête, (4) estimer la direction des yeux par analyse spatiale du centre de l’iris.

Cette approche est particulièrement attractive pour une application embarquée : elle est rapide, légère, interprétable et nécessite très peu de données d’entraînement.

Conclusion : Elle semble être l’approche la plus adaptée au projet (robustesse, simplicité, coût faible, temps réel).

3.12.3 Benchmark technologique

Nous comparons ici trois bibliothèques ou modèles majeurs :

- **MediaPipe FaceMesh (Google Research)**
- **OpenFace 2.0 (CMU)**
- **YOLO-Face (basé sur YOLOv5/YOLOv8)**

MediaPipe FaceMesh MediaPipe FaceMesh [14] propose un maillage facial de 468 points, dont des points dédiés à l’iris, permettant une estimation fine de la direction des yeux.

Avantages :

- Très rapide, optimisé pour CPU
- Intègre naturellement les landmarks des yeux et de l’iris
- Très robuste en conditions réelles

Inconvénients :

- Pas de vecteur de regard directement fourni (calcul géométrique nécessaire)

OpenFace 2.0 OpenFace 2.0 [1] est une bibliothèque académique open-source dédiée à l’analyse faciale (landmarks, action units, head pose, regard). Elle repose sur un pipeline qui n’est pas optimisé pour les CPU légers comme des mobiles.

Avantages :

- Très complet (AUs, head pose, eye gaze)
- Référence académique solide

Inconvénients :

- Semble plutôt lourd pour un système embarqué léger

YOLO-Face YOLO-Face est un dérivé des architectures YOLOv5/v8 optimisées pour la détection rapide de visages [5]. Bien qu’efficace, il ne fournit pas directement les landmarks nécessaires pour l’analyse de l’iris.

Avantages :

- Très rapide en détection de visage
- Simple à intégrer

Inconvénients :

- Ne fournit que la boîte englobante du visage
- Landmarks limités ou inexistants selon la version

Compte tenu :

- des limitations computationnelles d'un véhicule embarqué,
 - de la nécessité d'un système robuste en conditions lumineuses variées,
 - de la précision nécessaire uniquement pour classer des zones de regard,
- nous retenons l'approche :

MediaPipe FaceMesh + estimation géométrique du regard

Ce choix permet :

- un fonctionnement temps réel optimisé,
- une indépendance vis-à-vis des datasets calibrés,
- une grande robustesse inter-individuelle.

Un **Plan B** reste envisagé : entraîner un petit modèle supervisé (MLP ou SVM) sur les landmarks des yeux afin de corriger les erreurs dans certains cas limites.

3.12.4 Conclusion

Le benchmark technologique met clairement en évidence que les méthodes basées sur les landmarks constituent le meilleur compromis pour un système embarqué de suivi du regard pour la conduite automobile. MediaPipe FaceMesh se distingue nettement par sa rapidité, sa robustesse et la disponibilité des points d'iris nécessaires à une estimation géométrique fiable.

Les approches alternatives (OpenFace et YOLO-Face) présentent des performances remarquables dans leurs domaines respectifs, mais ne répondent pas aux exigences de légèreté, de vitesse et de stabilité imposées par ce projet.

4 Perception extérieure

4.1 État de l'art

La navigation autonome est traditionnellement décomposée en trois phases distinctes : la décision de la manœuvre à adopter, la planification de trajectoire, et l'exécution. Pour que les modules de décision et de planification fonctionnent, ils nécessitent une représentation précise de l'espace navigable. Dans ce contexte, la segmentation de la route par LiDAR constitue la brique fondamentale du système.

4.2 Prétraitement des données

L'étape de prétraitement constitue le maillon critique entre le flux de données LiDAR asynchrone issu de ROS 2 Jazzy et l'architecture neuronale. Dans ce projet, le prétraitement ne se limite pas à un simple formatage, mais inclut un nettoyage spatial et une structuration en "Pillars" optimisée pour l'inférence en temps réel sur GPU.

4.2.1 Acquisition et Normalisation de l'Intensité

Les données brutes sont capturées via le topic `/points` sous la forme de messages `sensor_msgs/PointCloud2`. Chaque point est initialement défini par ses coordonnées (x, y, z) et son intensité de réflexion r .

Le script effectue une vérification de la plage dynamique de l'intensité. Si $r_{max} > 1.0$, une normalisation linéaire est appliquée :

$$r_{norm} = \frac{r}{255} \quad (1)$$

Cette étape est essentielle pour assurer la stabilité numérique des poids du réseau lors de la phase d'apprentissage et de l'inférence.

4.2.2 Filtrage Spatial et Centrage Dynamique

Pour réduire la charge de calcul et éliminer les points non pertinents pour la segmentation de la route (arbres, bâtiments, bruit de capteur), deux filtres sont appliqués :

- Filtre de Hauteur (Z-Filtering) : Seuls les points respectant la condition $z \in [-2.5, 0.5]$ mètres sont conservés. Cela permet d'isoler la surface de roulement.
- Translation de Scène : Afin de maintenir le véhicule au centre de la zone de détection, nous appliquons une transformation relative basée sur la moyenne des points reçus :

$$x_{trans} = x_{raw} - \bar{x} + 35.0 \quad (2)$$

$$y_{trans} = y_{raw} - \bar{y} \quad (3)$$

4.2.3 Génération des Pillars (Voxelisation 2D)

Le nuage de points filtré est ensuite discrétisé en une grille 2D à l'aide de l'utilitaire `PointToVoxel`. Contrairement à une voxelisation 3D classique, les "Pillars" (piliers) sont des voxels dont la hauteur est égale à la plage totale de l'axe Z .

Les paramètres configurés dans notre implémentation sont les suivants :

Paramètre	Valeur
Taille des Pillars (V_{size})	$0.16m \times 0.16m \times 4m$
Plage de calcul (X, Y, Z)	$[0, 69.12], [-39.68, 39.68], [-3, 1]$
Nb. max de points par Pillar (N)	32
Nb. max de Pillars (P)	16 000

TABLE 1 – Paramètres de la grille de Pillars

4.2.4 Enrichissement des Caractéristiques (Feature Augmentation)

Chaque point à l'intérieur d'un pilier est enrichi pour capturer la géométrie locale. Au lieu des 4 dimensions initiales, chaque point est transformé en un vecteur $D = 9$ par le module `PillarVFE` (Voxel Feature Extractor). Ce vecteur inclut :

- Les coordonnées originales : (x, y, z, r) .
- La distance relative au centre arithmétique des points du pilier : (x_c, y_c, z_c) .
- Le décalage par rapport au centre géométrique de la cellule : (x_p, y_p) .

4.2.5 Formation du Tenseur d'Entrée

Le résultat final de ce prétraitement est la création d'un dictionnaire de batch (`batch_dict`) contenant :

1. `voxels` : Tenseur des caractéristiques de forme (P, N, D) .
2. `voxel_coords` : Coordonnées des piliers sur la grille 2D.
3. `voxel_num_points` : Nombre de points réels par pilier (pour ignorer le padding).

L'ensemble de ces données est transféré vers la mémoire CUDA pour permettre le traitement parallèle par le modèle.

4.3 Architecture du Modèle et Entraînement

Cette subsection détaille la configuration structurelle du réseau PointPillars ainsi que la méthodologie d'apprentissage supervisé mise en place à partir de données réelles capturées sur la plateforme ZOE.

4.3.1 Détails de l'Architecture Neuronale

Le modèle **PointPillarsRoadSegmentation** repose sur une adaptation de l'architecture originale pour la segmentation sémantique en vue de dessus (BEV).

Pillar Feature Net (VFE) Le module **PillarVFE** est la première couche apprenante. Elle traite le tenseur de piliers de dimension $(P, N, 9)$:

- Linéarisation : Les 9 caractéristiques de chaque point sont projetées dans un espace de dimension supérieure (généralement 64) via une couche linéaire (**Linear**), suivie d'une **BatchNorm** et d'une activation **ReLU**.
- Max Pooling : Pour chaque pilier, seule la caractéristique maximale parmi les N points est conservée. On passe d'une structure de points à une structure de caractéristiques de piliers.

Backbone 2D et Tête de Segmentation Une fois les piliers projetés sur une grille 2D de 432×496 par le module **Scatter**, le **BaseBEVBackbone** traite cette "image" :

- Blocs de Convolution : Le backbone utilise des séries de couches **Conv2d** avec des strides de 2 pour réduire la résolution spatiale tout en augmentant la profondeur sémantique.
- Multi-Scale Fusion : Les sorties de différents niveaux de résolution sont ré-échantillonnées et concaténées, formant un bloc de 384 canaux.
- SegHead (La tête de segmentation) : Elle se compose de :
 1. Une convolution 3×3 ($384 \rightarrow 128$ canaux).
 2. Un Upsampling bilinéaire (facteur 2) pour affiner les contours de la route.
 3. Une convolution 3×3 ($128 \rightarrow 64$ canaux).
 4. Une convolution finale 1×1 ($64 \rightarrow 1$ canal) produisant les logits de segmentation.

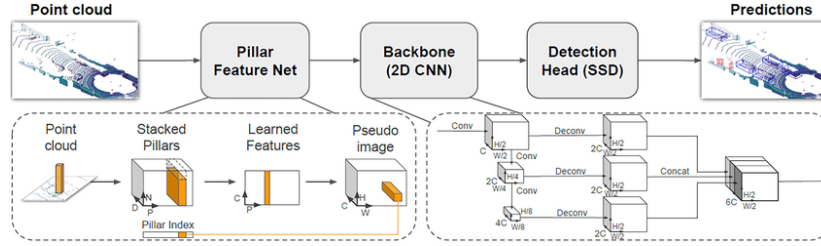


FIGURE 2 – Schéma du fonctionnement de PointPillars

4.3.2 Stratégie d'Entraînement et Dataset

Afin d'entraîner ce modèle nous avons du utiliser les bags fournis au début du projet. Cependant, ces données n'étaient pas exploitables en tant que telles. Il a fallut les extraire et les traiter.

Génération de la Vérité Terrain (Ground Truth) Le script **DataExtractorNode** a permis de constituer le dataset en synchronisant deux sources d'informations :

- Entrée (X) : Le nuage de points brut issu du topic **/points**.
- Cible (Y) : Les points segmentés manuellement publiés sur les topics **/road** (classe positive) et **/nonroad** (classe négative).

Pour chaque frame, les points de **/road** sont projetés sur une grille BEV de résolution $0.16m$. Un masque binaire (**.png**) est alors généré par projection spatiale, complété par une dilatation morphologique (noyau 3×3) pour assurer la continuité de la surface de la route.

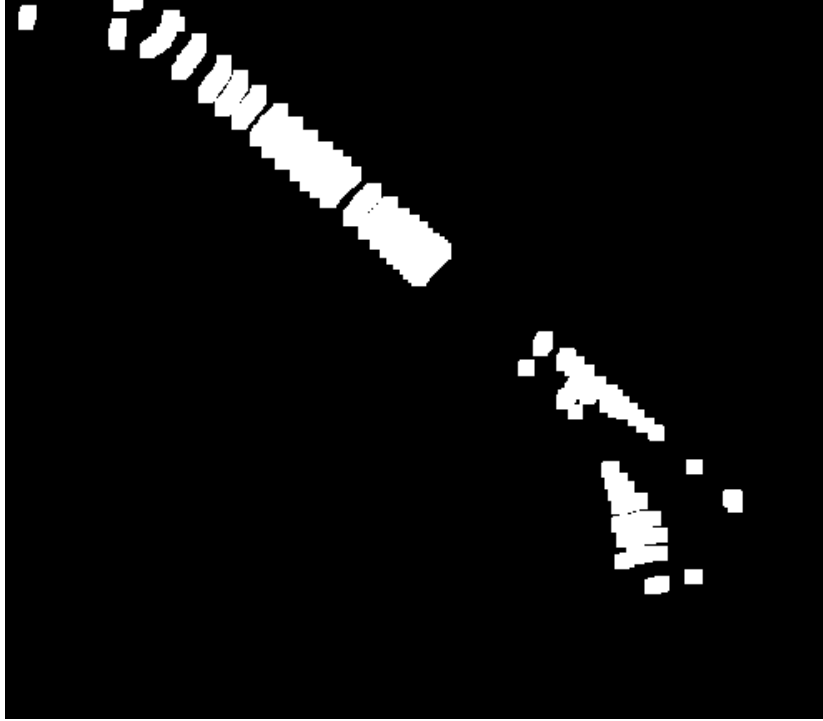


FIGURE 3 – Masque crée à partir du bag

Paramètres de la session d'entraînement L'entraînement a été effectué *from scratch* pour garantir que le modèle apprenne spécifiquement les caractéristiques du LiDAR Pandora utilisé sur la ZOE :

- Volume : Extraction de plusieurs centaines de paires (Nuage de points / Masque).
- Optimiseur : Adam avec un *learning rate* de 10^{-4} .
- Fonction de perte : `BCEWithLogitsLoss`, calculant l'erreur pixel par pixel entre la prédiction et le masque de vérité terrain.
- Durée : 100 époques avec des sauvegardes intermédiaires toutes les 20 époques pour monitorer la convergence et pouvoir tester les différents poids intermédiaires.

```

1 # Calcul de la perte sur le masque BEV
2 output = model(batch_dict).squeeze(1)
3 loss = criterion(output, target.to(device))
4 loss.backward()
5 optimizer.step()

```

Listing 1 – Boucle d'entraînement simplifiée

A la fin de cet entraînement et après test des différents fichiers de poids nous en sommes venus à la conclusion que le fichier de poids le plus adapté était celui de la 80ème époque.

4.4 Post-traitement de la Segmentation

Une fois que le modèle a généré la carte de caractéristiques en Bird's Eye View (BEV), une étape de post-traitement est indispensable pour convertir les probabilités en un masque binaire pour la navigation du véhicule.

4.4.1 Conversion Probabiliste et Seuillage

La sortie finale du réseau (`seg_head`) fournit des "logits". Nous appliquons une fonction Sigmoidale pour transformer ces valeurs en probabilités $P \in [0, 1]$ pour chaque pixel de la grille :

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (4)$$

Un seuil de confiance rigoureux (`threshold = 0.92`) est appliqué. Ce choix privilégie la précision (éviter les faux positifs) garantissant que les zones marquées comme "route" sont hautement probables.

4.4.2 Filtrage Morphologique et Lissage

Cependant, même avec cette mesure on se retrouve avec un bruit assez important. Pour éliminer le bruit résiduel, nous appliquons une chaîne de traitement d'image via la bibliothèque **OpenCV** :

1. Ouverture Morphologique : Utilisation d'un noyau 2×2 pour supprimer les petits points de bruit blanc (pixels isolés activés par erreur).
2. Flou Gaussien : Application d'un filtre avec un noyau 3×3 pour lisser les contours de la route et réduire l'aliasing dû à la voxelisation.
3. Re-seuillage : Une opération finale de seuillage binaire à 128 (sur 255) permet de fixer définitivement les frontières de la zone segmentée.

4.4.3 Masquage Géométrique et Zones d'Exclusion

L'une des particularités de notre implémentation est l'ajout d'un masque géométrique fixe via la fonction `get_combined_mask`. Ce masque intervient pour corriger les erreurs et le bruit là où l'information n'est pas absolument nécessaire :

- Zones d'exclusion latérales (Triangles) : Deux triangles sont dessinés à gauche et à droite de la base du véhicule (`exclusion_dist` et `exclusion_base`). Ils permettent de supprimer les détections aberrantes sur les côtés proches.
- Couloir d'exclusion (Rectangles) : Nous définissons un corridor central (`corridor_width` et `corridor_length`) à l'arrière et à l'avant immédiat du capteur.

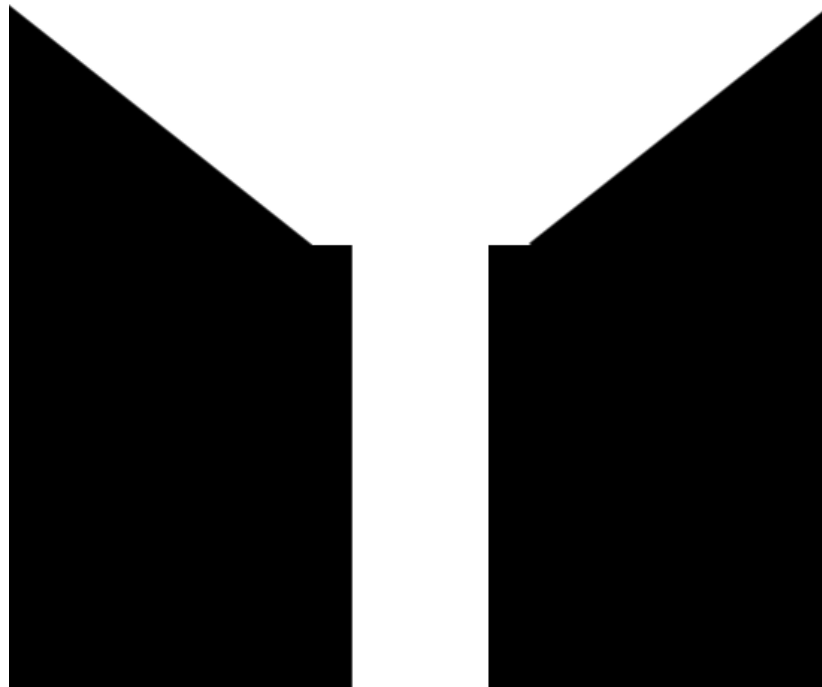


FIGURE 4 – Masque d'exclusion géométrique final

Ainsi à la suite de toutes ces étapes, on obtient un flux d'image en noir et blanc comme ci-dessous.

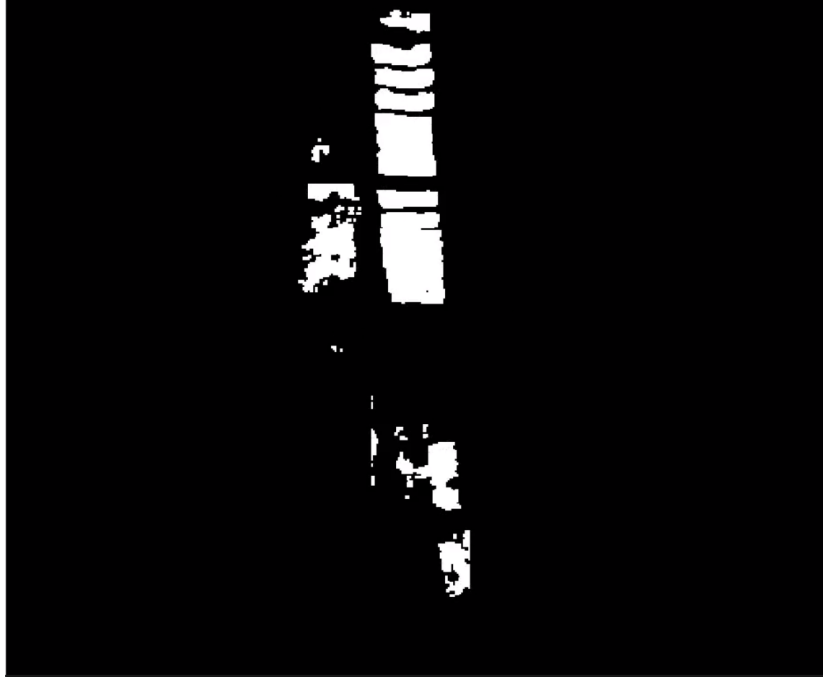


FIGURE 5 – Résultats obtenus après inférence et post-traitement

4.4.4 Gestion de la Latence et Publication

Le nœud ROS 2 intègre un système de monitoring de la performance. Le temps écoulé entre la réception du nuage de points et la publication du masque est calculé systématiquement :

$$T_{latence} = (t_{fin} - t_{debut}) \times 1000 \text{ ms} \quad (5)$$

Cette valeur est publiée sur le topic `/perception/latency`. L'utilisation du profil de QoS `sensor_data` (Best Effort) permet d'assurer que le flux d'images `/perception/road_mask` ne sature pas la bande passante du réseau de la ZOE.

4.4.5 Conclusion

Cependant malgré les différentes étapes de post-traitement il reste quand même un bruit résiduel important sur certaines frames. Il est donc possible d'améliorer cet aspect de la perception. De plus le réseau neuronal n'est pas très précis et sur certaines frames on retrouve des "taches noires" sur des endroits qui sont censés être de la route. Ainsi le module de perception reste à améliorer.

4.5 Organisation de l'Environnement

Le déploiement d'un modèle de Deep Learning tel que PointPillars sur le système embarqué de la ZOE nécessite un environnement d'exécution précis. Cette section détaille les solutions retenues pour garantir la compatibilité matérielle et la fluidité des communications.

4.5.1 Isolation des Environnements avec Venv

La diversité des bibliothèques nécessaires (PyTorch, spconv, OpenPCDet) et leurs dépendances spécifiques vis-à-vis de CUDA imposent une isolation stricte. Nous utilisons un environnement Venv appelé `pcdet_env` et dédié à l'inférence neuronale. Il contient les versions spécifiques de Python et de torch.

4.5.2 Middleware Zenoh : Pont de Compatibilité

Un problème majeur est apparu lors de l'intégration : l'incompatibilité entre les drivers NVIDIA requis par le modèle et ceux installés sur la ZOE.

Pour pallier ce problème, nous avons implémenté Zenoh comme pont de communication. Zenoh offre plusieurs avantages.

4.5.3 Automatisation et Alias

Afin de simplifier le déploiement sur le terrain et d'assurer la répétabilité des tests, une série d'alias a été configurée. Ces commandes permettent de :

1. Sourcer automatiquement les espaces de travail.
2. Activer l'environnement Venv approprié.
3. Lancer le build colcon.

4.5.4 Configuration du Shebang et Exécution

L'utilisation d'un shebang spécifique est nécessaire afin d'avoir les packages nécessaires à l'exécution du node :

```
1 #!/home/younes/pcdet_env/bin/python3
```

5 Localisation

La localisation consiste à estimer la pose du véhicule, c'est-à-dire sa position et son orientation dans le plan, en combinant plusieurs sources d'information. Dans notre projet, elle sert principalement à déterminer précisément où se trouve la voiture sur le circuit de Séville afin d'informer correctement le module de Navigation.

5.1 Contexte général et méthodes de localisation

De manière générale, la localisation dans les véhicules intelligents s'appuie sur l'estimation dynamique de la pose à partir de capteurs variés et de données géoréférencées. Dans les approches, on peut utiliser des cartes préexistantes ou bien construire une carte inconnue tout en se localisant à l'intérieur de cette carte, en même temps, en boucle (SLAM). Deux grandes familles de méthodes existent aujourd'hui [4] :

- **Les méthodes par filtrage**, comme les filtres de Kalman non linéaires ;
- **Les méthodes par optimisation et graphes de contraintes**, qui permettent notamment des corrections lors des fermetures de boucle.

Les systèmes complets combinent souvent :

- des capteurs de localisation à l'estime (centrale inertielle, odométrie ou vision),
- et des capteurs de localisation absolue comme un GNSS différentiel, ou encore la perception de panneaux et marquages géoréférencés.

Dans notre cas, le circuit étant fixe et connu à l'avance, nous adoptons une stratégie de **localisation aidée par la carte** plutôt que du SLAM.

5.2 Objectif de la localisation dans notre système

Le rôle du module Localisation est d'estimer la pose (x, y, θ) du véhicule à partir de trois sources principales :

- une carte statique du circuit, trouvée sur OSM et que nous avons modifié légèrement, segmentée en différentes zones ;
- la position GNSS, donnée par un capteur Septentrio, utilisée pour identifier la zone du circuit et réduire le risque de confusion ;
- un nuage de points issu de la Perception Extérieure avec le Lidar, contenant les zones conduisibles et non conduisibles.

Cette fusion permet d'assurer que le véhicule se situe correctement sur le bon segment du tracé et d'alimenter la prise de décision du module Navigation.

5.3 Prétraitement des données

Afin de rendre le map-matching robuste et rapide, plusieurs étapes de prétraitement sont appliquées :

Projection du LiDAR en 2D. Le nuage 3D est projeté dans un plan (x, y) en supprimant la dimension verticale. Cela permet de travailler en 2D, ce qui correspond à la structure du circuit. Le module de localisation reçoit ce nuage sous le format d'une image mono8 en niveau de gris. Les pixels clairs représentent la route détectée.

Normalisation des échelles. Si la carte OSM convertie en nuage et les données LiDAR n'ont pas exactement la même échelle, un facteur d'échelle fixe est appliqué.

Le résultat est un nuage 2D propre et prêt pour l'alignement.

5.4 Map-matching : principes et méthodes retenues

Le cœur de la localisation repose sur le **map-matching**, c'est-à-dire l'alignement entre :

- une carte 2D du circuit, représentée sous forme de tracé segmenté extrait d'OpenStreetMap ;
- le nuage de points fourni par le lidar après classification extérieur/route.

5.4.1 Découpage en segments via GNSS

La carte est découpée en plusieurs sections (ligne droite, rond-point haut, rond-point bas). Il s'agit d'une polyligne.

Nous appelons d'abord le GNSS et convertissons les coordonnées dans le repère ENU du circuit. Puis les coordonnées sont utilisées pour faire une sélection des segments du circuit les plus proches, ce qui nous donne une polyligne plus petite.

A partir de la deuxième localisation GNSS appelée, la différence entre celle-ci et la précédente est faite pour se donner une orientation (theta) initiale.

5.4.2 Alignement géométrique via ICP

Une fois la polyligne sélectionnée, nous appliquons un algorithme d'alignement géométrique pour estimer la pose précise dans ce segment. Deux méthodes sont particulièrement pertinentes ICP et NDT, toutes deux mentionnées dans le rapport d'études mais dans un souci computationnel, seul ICP a été retenu. NDT étant une méthode trop demandante pour le matériel utilisé :

- **ICP (Iterative Closest Point)** : recherche itérative de la transformation (R, t) qui minimise la distance entre le nuage lidar projeté en 2D et la carte [12]. L'algorithme résout à chaque itération :

$$\arg \min_{R, t} \sum_i \|Rp_i + t - q_{\pi(i)}\|^2$$

où :

- p_i : point i du nuage de points lidar à aligner,
 - $q_{\pi(i)}$: point correspondant sur la carte (via correspondance $\pi(i)$),
 - R : matrice de rotation 2D (ou 3D selon contexte),
 - t : vecteur de translation,
 - $\pi(i)$: fonction de correspondance qui associe chaque point p_i à un point $q_{\pi(i)}$ de la carte.
- ICP suit un cycle : correspondances, estimation, application, convergence. Il est simple et rapide mais sensible aux mauvaises initialisations — ce que les coordonnées GNSS corrige en nous donnant une pose initiale mais également une sélection de polyligne réduite à un rayon de 5 mètres autour des coordonnées GNSS, ce qui évite de chercher sur toute la carte. Pour chaque itération, l'algorithme identifie les correspondances entre les points du nuage cible et ceux du nuage source. Il calcule ensuite la transformation qui minimise l'erreur quadratique moyenne.

Une fois celle-ci trouvée il renvoie la matrice de rotation et de translation nécessaire pour aligner les 2 nuages.

Il est important de mentionner 2 points dans notre cas :

- **Premièrement**, notre carte étant une polygline, cet algorithme ne s'applique pas. Il nous faut un nuage de points cible.
Avant l'exécution de l'ICP, un prétraitement est effectué qui prend la polygline sélectionnée et génère un nuage de points.

Pour cela, des points sont générés de manière dense sur les lignes droites qui relient les points de la polygline. Ainsi que sur les côtés, parallèlement à ces lignes, jusqu'à l'épaisseur de la route des deux côtés.

La route a donc été mesurée, et nous avons modifié notre polygline pour s'assurer qu'elle correspondait au centre de la route.
En faisant cela, nous nous assurons que l'ICP puisse fonctionner comme prévu.

- **Deuxièmement**, le nuage de points en provenance du Lidar est dans son propre repère, qui est métrique.
L'algorithme doit nous renvoyer une pose dans le repère ENU. Il faut donc renvoyer les coordonnées du point de la carte qui a été trouvé comme correspondant avec l'origine du lidar.
Pour l'orientation, il suffit de prendre le theta de la matrice de rotation.

Afin d'optimiser le temps de calcul, un KD-tree est également utilisé, il s'agit d'un arbre binaire de rangement qui trie en fonction des coordonnées de l'espace les points.
Cela aide l'ICP à converger plus rapidement vers des correspondances vraisemblables, puisque les points sont ordonnés, il évite donc de tester toutes les combinaisons possibles, même absurde.

Il devient donc pertinent, afin de rester sur un temps de calcul raisonnable, de limiter le nombre d'itérations maximum de l'ICP.

Il est possible d'affiner encore le résultat obtenu avec de l'odométrie, en prenant en considération les coordonnées précédemment obtenues, ainsi que l'orientation estimée. Mais il y a également une possibilité de fusion avec des filtres de Kalman pour plus de robustesse.

5.5 Sortie du module Localisation

Le module Localisation transmet donc en sortie :

- la pose estimée du véhicule (x, y, θ) ;
- la vitesse estimée, calculée en faisant la différence dans le temps et l'espace de deux poses estimées successives.

Dans le cadre du projet, nous avons utilisé la SPAN afin de comparer les valeurs obtenues de la pose et observer la performance.

Cette information alimente ensuite la Navigation et permet au système de suivre précisément le tracé du circuit.

6 Navigation

6.1 État de l'art

La navigation comporte généralement trois phases : la décision de la manœuvre à adopter, la planification de trajectoire, et l'exécution [4]. L'exécution de la trajectoire concerne le module Contrôle, nous allons donc ici préciser uniquement les parties décision et planification.

6.1.1 Décision de la manœuvre à adopter

La partie décision consiste, grâce à la détection de l'environnement et à la pose du véhicule, à décider de la manœuvre à adopter. Par exemple, si la voiture détecte un cycliste, elle peut décider de le dépasser, ou rester sur sa voie si un autre véhicule arrive sur la voie d'en face. Cette partie ne nous intéresse pas dans le projet, puisque nous n'aurons a priori qu'une

seule manœuvre à effectuer : suivre la route. Nous citerons tout de même brièvement quelques méthodes utilisées dans certains travaux de recherche : la modélisation par apprentissage [2], la modélisation probabiliste avec processus de décision markovien [10], ou encore les systèmes experts [4].

6.1.2 Planification de trajectoire

Pour faire de la navigation et de la planification de trajectoires, plusieurs méthodes ont déjà utilisées dans plusieurs travaux de recherche. Comme pour la partie précédente sur la décision, nous n'aurons a priori pas besoin de cette partie, puisque nous pensons nous servir uniquement de tracés de la route fournis à l'avance. Nous présenterons tout de même plus en détails ces méthodes que nous pourrions implémenter si nous avons le temps.

Pour calculer un chemin de consigne, il est possible de représenter l'espace navigable par un graphe, puis d'appliquer un algorithme de recherche de chemin dans un graphe pour calculer une trajectoire, qu'il est possible de lisser. Pour la représentation sous forme de graphe, il est possible de faire une décomposition de Dirichlet, une décomposition en cellules, ou un graphe de visibilité [4]. Afin de parcourir le graphe, plusieurs algorithmes sont souvent cités, comme l'algorithme de Dijkstra, A*, ou des algorithmes non déterministes comme RTT ou génétique [4]. Un autre algorithme, BMSSP a été découvert plus récemment et est théoriquement plus rapide que Dijkstra ou A* [7]. Cependant, une implémentation Python donnée sur GitHub [6] nous montre que cet algorithme peut être difficile à implémenter et à optimiser dans le cadre de notre projet, et qu'une simple implémentation de l'algorithme de Dijkstra suffit largement.

6.2 Le module Navigation dans notre projet

6.2.1 Entrées du module Navigation

Le module Navigation du système recevra plusieurs entrées. D'une part, la pose du mobile envoyée par le module Localisation. D'autre part, la commande de direction envoyée par le module Perception Intérieure. En plus de ces deux inputs, le module pourra obtenir des mesures de la vitesse de la voiture via les capteurs (vitesses des quatre roues ainsi que la vitesse angulaire au centre de la voiture [à vérifier]). Enfin, le module aura accès au tracé de la route ainsi qu'à la carte, comme le module Localisation.

6.2.2 Fonctionnement du module Navigation

Grâce à une pose précise, à la commande de direction, et à la carte, le rôle du module sera de déterminer deux choses : sur quel type de route se trouve la voiture (entrée ou sortie de rond-point, intérieur d'un rond-point, autre route), ainsi que la trajectoire à suivre. Séville étant composée de deux ronds-points et d'une ligne droite, nous découperons le tracé de la route en trois parties : une pour chaque rond-point et une pour la ligne droite et l'extérieur des ronds-points, comme montré en Figure 9.

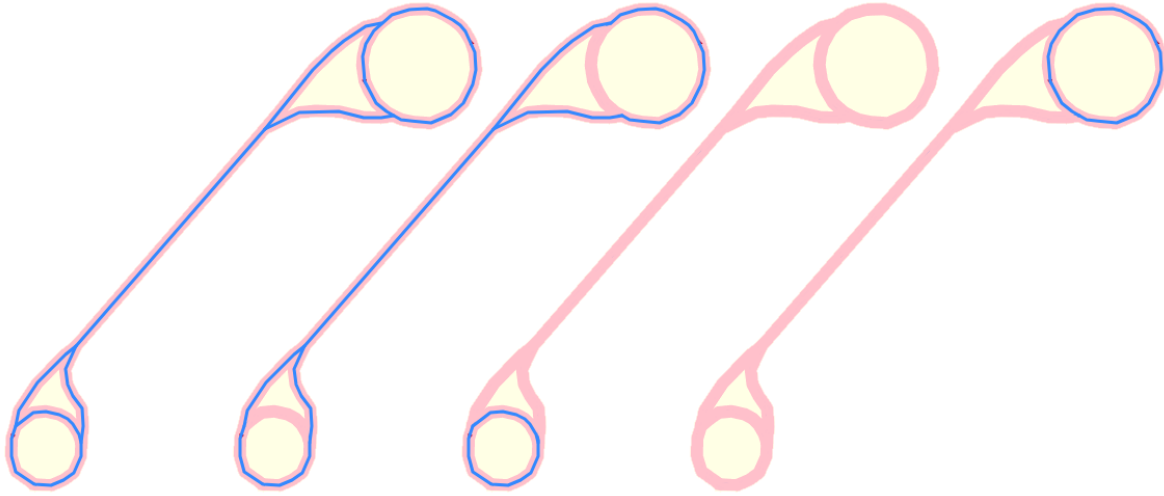


FIGURE 6 – Les tracés de la route de Séville. De gauche à droite : tracé complet, ligne droite et extérieurs des ronds-points ("Ligne"), rond-point bas ("RP Bas"), rond-point haut ("RP Haut"). Ces tracés ont été repris d'OpenStreetMap et retouchés.

Par exemple, si la voiture se trouve en sortie de rond-point (information obtenue grâce à la pose donnée par le module Localisation, ainsi qu'à la carte qui contiendra les différentes zones), et que le conducteur a envoyé la commande de direction "Droite", alors la voiture devra suivre le tracé "Ligne" pour sortir du rond-point. Au contraire, si le conducteur indique "Tout droit", ou "Gauche", la voiture restera sur le tracé du rond-point correspondant pour rester dans le rond-point.

6.2.3 Sorties du module Navigation

En sortie, le module enverra simplement la trajectoire à suivre (un des trois tracés) au module Contrôle.

7 Contrôle

7.1 Le module Contrôle dans notre projet

Le module Contrôle reçoit séparément deux informations en entrée de la part du module Navigation :

- L'indice du tracé à suivre parmi Rond-point Haut, Rond-point bas, Ligne droite, ou Rien (s'arrêter dans ce cas).
- La pose et la vitesse du véhicule, transmises à la Navigation depuis la Localisation (ou la span).

Lorsque le module reçoit le tracé à suivre, le tracé utilisé pour les calculs des lois de commande est changé en fonction (utilisé pour calculer l'écart latéral, angulaire et la courbure de la route). Si l'indice reçu ne correspond à aucun tracé, la vitesse désirée est mise à $0m/s$ et le véhicule s'arrête. Ce cas arrive lorsque l'utilisateur fait le geste "Stop" ou n'est pas concentré.

Lorsque le module reçoit la pose et la vitesse du véhicule, celui-ci calcule les nouvelles sorties des lois de commandes latérales et longitudinales (décrites plus bas).

En sortie, le module envoie de nombreuses informations concernant l'état du véhicule : pose, segment le plus proche, résultats des lois de commande, etc. Ces informations sont utiles pour l'affichage et le débogage. Le module utilise également l'accélération et l'angle au volant calculés pour les envoyer à la voiture.

7.2 Contrôle latéral

Pour la mise en oeuvre du contrôle latéral, nous avons utilisé une loi de commande Proportionnelle Dérivée avec action sur la courbure de la route κ [3] :

$$\delta = K_e \cdot \frac{L}{v^2} \cdot (e_d - e) + K_p \cdot \frac{L}{v} \cdot \text{submodpi2}(\alpha, \theta) + K_c \cdot L \cdot \kappa \text{ et } v \neq 0$$

Avec L la longueur du modèle, donc l'empattement de la voiture ; v la vitesse du véhicule ; $e_d - e$ l'erreur latérale par rapport au segment le plus proche ; $\text{submodpi2}(\alpha, \theta)$ l'écart angulaire entre le segment le plus proche et le véhicule ; δ l'angle des roues.

Les résultats ont mis longtemps à être concluants, il s'avérait en réalité que c'était principalement les gains des différentes composantes de la loi de commande qui n'étaient pas bien réglés. Nous avons changé les gains comme ceci, ce sont eux qui ont donné les meilleurs résultats en pratique :

$$K_e = 0.1, K_p = 0.5, K_c = 0.0$$

Le gain de courbure K_c est nul car en réalité le terme sur la courbure n'a que très peu d'impact sur le résultat final, et nous n'avons pas eu le temps de tester avec d'autres gains.

Nous utilisons également de l'avance de phase pour calculer l'écart, la pente, et la courbure au niveau de l'essieu avant plutôt qu'à l'essieu arrière de la voiture (qui correspond à l'origine du repère voiture). Enfin, dans le calcul de la loi de commande, nous n'utilisons pas directement la vitesse, nous avons ajouté un coefficient 0.5 pour que la commande soit plus réactive (notamment dans certaines parties des rond-points).

7.3 Contrôle longitudinal

Pour le contrôle longitudinal, nous avons simplement utilisé une loi de commande Proportionnelle Intégrale avec les gains suivants :

$$a = K_\varepsilon \cdot \varepsilon + K_I \cdot \int_0^t \varepsilon(\tau) d\tau$$

$$K_\varepsilon = 4.0, K_I = 0.4$$

Avec a l'accélération et $\varepsilon = v_d - v$ l'écart de vitesse du véhicule.

Là aussi, ce sont les gains qui ont donné les meilleurs résultats en pratique, même s'il en existe sûrement des meilleurs. Nous avons aussi saturé la valeur de l'intégrale dans l'intervalle $[-5, 5]$ pour ne pas que celle-ci prenne des valeurs trop importantes : par exemple, à l'arrêt, l'intégrale continue de croître donc la commande d'accélération devient de plus en plus grande. De plus, comme prévu, nous utilisons la courbure locale pour réduire la vitesse dans les rond-points en mettant un plafond à l'accélération latérale ($0.8m/s^2$) et en utilisant le lien suivant entre vitesse longitudinale et accélération latérale :

$$a_{lat} = v^2 \kappa$$

Nous n'avons pas utilisé d'action dérivée comme nous l'avions initialement prévu car ça ne semblait pas nécessaire au vu des résultats assez bons du contrôleur longitudinal. , et nous utilisons une vitesse maximale de 10km/h au lieu de 30km/h ou 15km/h.

7.4 Tracé

Nous avons rencontré des problèmes concernant le tracé utilisé. Pour rappel, nous pensions utiliser le tracé fourni par OpenStreetMap pour réaliser trois tracés (polyligne) à suivre : Rond-point haut, Rond-point bas et Ligne droite. Lors des phases de tests, le tracé s'est avéré très inexacte sur certaines parties de la route. En effet, le tracé est souvent trop proche des bords de la route, soit à gauche, soit à droite. Nous observons en Figure 7 que le tracé d'OpenStreetMap est parfois décalé avec la vérité terrain (jusqu'à environ 1 mètre), et que les entrées et sorties

des rond-points sont trop brusques et ne suivent pas bien la piste. Nous avons donc décidé d'améliorer le tracé d'OpenStreetMap en le rapprochant du tracé réel et en modifiant les entrées et sorties des rond-points à la main. Ce nouveau tracé permet une conduite plus stable sans sortie de route et améliore les résultats obtenus pour les deux lois de commande.

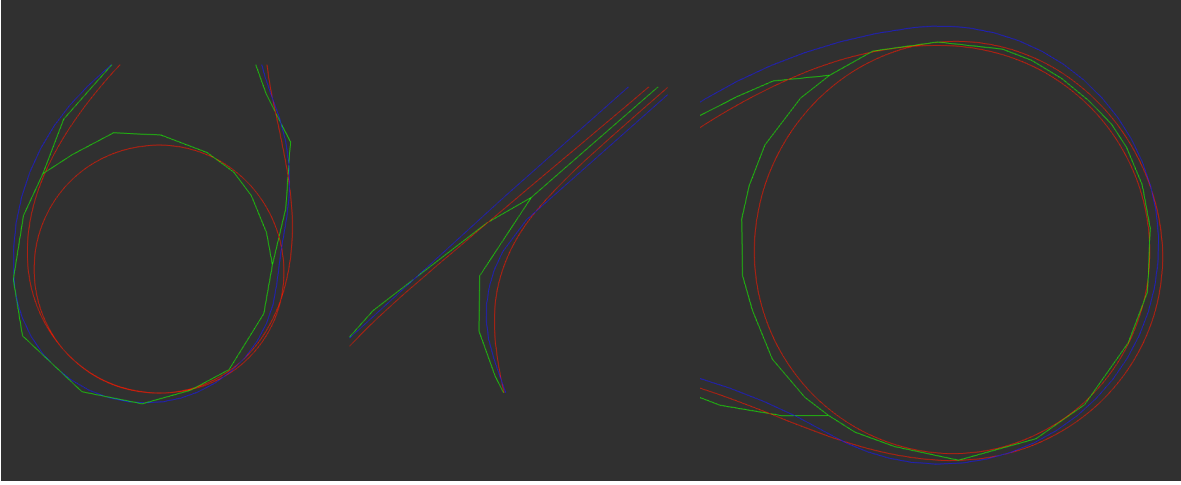


FIGURE 7 – Comparaison de trois tracés sur trois sections de Séville. En rouge, la vérité terrain obtenue grâce à un bag de la span. En bleu, le tracé SevilleCourbureSignee (utilisé uniquement lors des tests). En vert, le tracé obtenu de OpenStreetMap.

7.4.1 Résultats et Conclusion

Le contrôle, même si très loin d'être parfait, se dirige de manière plutôt satisfaisante. L'avance de phase et le meilleur tracé permettent un meilleur suivi de la route, tandis que les gains utilisés permettent de rendre la conduite plutôt lisse, notamment dans les rond-points, où le volant reste quasiment à un angle stable, tandis qu'il faisait des grands à-coups très brusques auparavant.

Lors des tests et de la démonstration finale, plusieurs problèmes nous sont apparus :

- Le contrôle latéral oscille doucement autour du tracé en ligne droite.
- La sortie du rond-point bas est difficile : le contrôle latéral tourne souvent trop tard et le conducteur de sécurité est obligé de ralentir pour ne pas percuter la barrière.

Concernant le premier problème, cela indique plusieurs choses, notamment que le contrôleur met du temps à converger. Les gains à $K_e = 0.1$ et $K_p = 0.5$ correspondent à des pôles proches de 0, donc à des temps de réponses assez longs, expliquant les oscillations. En effet, on a :

$$\begin{cases} K_e = s_1 s_2 = 0.1 \\ K_p = -s_1 - s_2 = 0.5 \end{cases} \implies \begin{cases} s_1 = \frac{-0.5 + i\sqrt{0.65}}{2} \\ s_2 = \frac{-0.5 - i\sqrt{0.65}}{2} \end{cases}$$

Nous aurions pu mettre des gains plus adaptés à un temps de réponse plus court mais les gains théoriques optimaux (les deux pôles à -2) donnaient des résultats pratiques très peu concluants et beaucoup trop brusques.

Le deuxième problème peut aussi s'expliquer par un contrôleur latéral assez lent, mais cela est aussi dû au tracé amélioré qui n'est pas optimal, ainsi que, probablement, au gain de courbure nul $K_c = 0$. En effet, une action sur la courbure aurait pu faire tourner le volant davantage dans cette situation, car la courbure à cet endroit de la route est assez élevée.

Le contrôle longitudinal quant à lui, est assez lisse et agréable, notamment en raison de la baisse de la vitesse maximale à 10km/h. L'arrêt du véhicule est un peu brusque mais fonctionne

correctement lorsque l'utilisateur fait le geste "Stop" ou regarde ailleurs.

8 Affichage

Le module Affichage a pour rôle de fournir un retour visuel complet à l'utilisateur afin de permettre : le suivi en temps réel du comportement du système, la validation expérimentale des modules de perception, localisation et navigation, et l'analyse post-expérimentation.

Il regroupe deux catégories d'informations : (1) l'état intérieur, issu de la perception intérieure ; (2) l'état extérieur, lié à la conduite et à la localisation.

Affichage des informations de conduite

L'affichage peut inclure plusieurs données issues directement des modules Localisation, Navigation et Contrôle :

- Position estimée du véhicule sur la carte statique du circuit, sous forme d'un marqueur mis à jour en temps réel.
- Orientation du véhicule (flèche ou repère).
- Vitesse actuelle.
- Trajectoire prévue, c'est-à-dire la portion du chemin planifiée dans les prochaines secondes par la Navigation.
- État de la navigation (normal, évitement, stop).
- Commandes appliquées : angle de direction, accélération/freinage (mode debug).

L'objectif principal est de superposer les données internes du système sur une carte statique afin d'observer directement la cohérence du map-matching et de la pose estimée.

Affichage des informations du contrôle

Le module Affichage récupère les données envoyées par le module Contrôle pour afficher en temps réel les éléments suivants, visibles en Figure 8

- La pose de la voiture
- L'angle des roues avant
- Le tracé suivi
- Le segment le plus proche

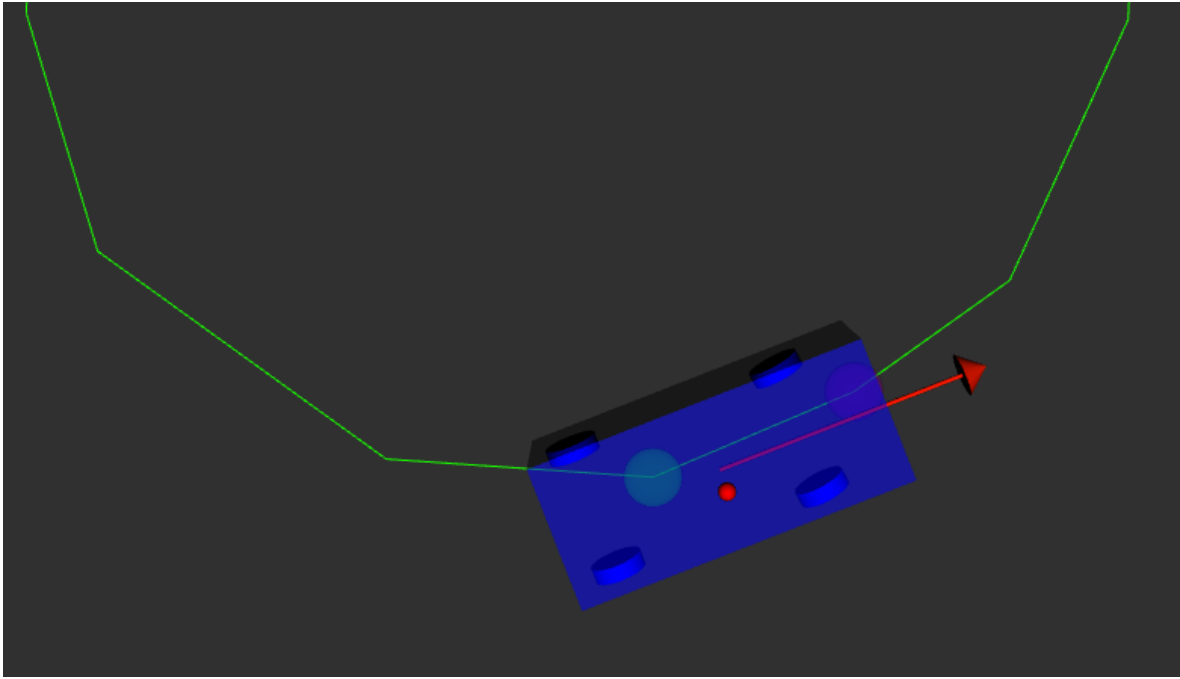


FIGURE 8 – Affichage du module Contrôle sur RViz. La voiture et ses roues sont en bleues. Le tracé suivi est en vert, avec un rond vert et un rond violet pour indiquer le début et la fin du segment le plus proche.

Affichage des informations de perception intérieure

Le module Affichage fournit aussi un retour explicite des décisions prises par la perception intérieure :

- Geste détecté (gauche, droite, haut, bas, ouverture, pince).
- Niveau de confiance du geste.
- Direction du regard : gauche, droite ou aucun, accompagnée de la confiance.
- Commande interne produite (geste ou regard retenu après fusion).

Cela permet de visualiser en temps réel la compréhension du conducteur par le système, indispensable lors de l'ajustement des seuils, des modèles de détection ou des règles de fusion. Ainsi que d'aider dans la phase de calibration des capteurs internes et dans les debuggages.

9 Organisation

9.1 Rôles

Voici l'organisation des rôles de notre projet :

- Chef de projet : Félix LIBURSKI
- Intégrateur : Younes EL MRABET
- Git Master : Dylan GIRAULT
- Architecte : Hugo HUYET-DUMONG

Voici également l'organisation des modules :

- Perception intérieure : Félix
- Perception extérieure : Younes
- Localisation : Hugo
- Navigation : Jules / Dylan
- Contrôle : Jules / Dylan
- Affichage : Hugo

9.2 Planning

FIGURE 9 – Planning prévisionnel de la répartition des tâches.

Références

- [1] Tadas BALTRUŠAITIS et al. “OpenFace 2.0 : Facial Behavior Analysis Toolkit”. In : *IEEE International Conference on Automatic Face and Gesture Recognition* (2018). URL : <https://github.com/TadasBaltrusaitis/OpenFace>.
- [2] Mariusz BOJARSKI et al. *End to End Learning for Self-Driving Cars*. 2016. arXiv : 1604.07316 [cs.CV]. URL : <https://arxiv.org/abs/1604.07316>.
- [3] Philippe BONNIFAIT. “Machines intelligentes (Poly de cours)”. In : (2025).
- [4] Philippe BONNIFAIT et Clément ZINOUNE. “Introduction aux techniques de navigation autonome pour les véhicules intelligents”. In : *Techniques de l’Ingénieur*. Technologies de l’information Réf : S7819 v1 (jan. 2021), p. 1-20. DOI : 10.51257/a-v1-s7819. URL : <https://hal.science/hal-03110973>.
- [5] Community CONTRIBUTORS. *YOLO-Face Documentation*. 2023. URL : <https://github.com/derronqi/yolov5-face>.
- [6] DARKTHYME. *SSSP Algorithms Comparisons*. https://github.com/DarkThyme/SSSP_Algorithms_Comparisons. 2025.
- [7] Ran DUAN et al. *Breaking the Sorting Barrier for Directed Single-Source Shortest Paths*. 2025. arXiv : 2504.17033 [cs.DS]. URL : <https://arxiv.org/abs/2504.17033>.
- [8] *MediaPipe Hand Landmarker*. Accessed 2025. URL : https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker.
- [9] *MediaPipe Hands — Documentation*. Accessed 2025. URL : <https://mediapipe.readthedocs.io/en/latest/solutions/hands.html>.
- [10] Frans A. OLIEHOEK et Christopher AMATO. “The Decentralized POMDP Framework”. In : *A Concise Introduction to Decentralized POMDPs*. Cham : Springer International Publishing, 2016, p. 11-32. ISBN : 978-3-319-28929-8. DOI : 10.1007/978-3-319-28929-8_2. URL : https://doi.org/10.1007/978-3-319-28929-8_2.
- [11] Google RESEARCH. *On-device, Real-Time Hand Tracking with MediaPipe*. 2019. URL : <https://research.google/blog/on-device-real-time-hand-tracking-with-mediapipe/>.
- [12] Szymon RUSINKIEWICZ et Marc LEVOY. “Efficient Variants of the ICP Algorithm”. In : *Proceedings of the Third International Conference on 3-D Digital Imaging and Modeling*. 2001, p. 145-152. DOI : 10.1109/IM.2001.924423.
- [13] *Ultralytics Hand Keypoints Dataset*. Accessed 2025. URL : <https://docs.ultralytics.com/datasets/pose/hand-keypoints/>.
- [14] Ming YONG. “MediaPipe Face Mesh”. In : 2020. URL : <https://github.com/google-ai-edge/mediapipe/wiki/MediaPipe-Face-Mesh>.